TacticalGrid

Sistema de búsqueda y decisión adversarial en un juego táctico 2D

Autor: Cristian Camilo Osorio Granada

Institución: Universidad de Caldas

Curso: Sistemas Inteligentes I

Año: 2026

1. Introducción

1.1 Contexto

Este proyecto parte de una pregunta que aparece de forma recurrente en el curso de Sistemas Inteligentes: ¿cómo cambia la forma de decidir de un sistema cuando pasa de encontrar una ruta, a optimizar su costo, a usar conocimiento heurístico, y finalmente a actuar frente a un adversario que también toma decisiones?

Cada una de esas etapas exige una herramienta distinta y, sobre todo, exige entender sus supuestos. Un algoritmo que garantiza el camino más corto puede ser una mala elección cuando lo que importa es el costo. Una heurística que funciona en un mapa abierto puede fallar cuando hay obstáculos. Una decisión que parece razonable en solitario puede resultar mala cuando otro jugador responde.

El proyecto intenta recorrer esas cuatro etapas sobre un mismo problema, sin cambiar de dominio, para que las diferencias entre algoritmos se puedan observar y medir directamente.

1.2 El problema

TacticalGrid simula un juego táctico 2D por turnos. Sobre una cuadrícula se ubican dos bandos, A y B, cada uno con varias unidades. En una zona neutral hay un recurso estratégico. Cada equipo intenta llegar hasta él, tomarlo y transportarlo hasta su propia base antes que el adversario.

El mapa tiene obstáculos y varios tipos de terreno, cada uno con un costo de tránsito distinto. Ese detalle es el que hace interesante el problema: la ruta con menos movimientos no siempre es la más económica.

El objetivo del proyecto no es desarrollar un videojuego. El juego es el entorno de experimentación; lo que se construye es el sistema que decide dentro de él.

1.3 Requisitos cubiertos

Algoritmos de búsqueda no informada: BFS y DFS.

Algoritmos basados en costos: búsqueda de costo uniforme (UCS).

Algoritmos informados: A* y búsqueda en haz (Beam Search), este último con memoria limitada.

Algoritmos adversariales: Minimax y Minimax con poda alfa-beta, con profundidad configurable.

Heurísticas: distancia Manhattan; Manhattan escalada por el costo mínimo del terreno; y una heurística basada en la distancia real en pasos calculada con un recorrido en anchura desde el objetivo. Las distancias euclidiana y Chebyshev quedan documentadas pero no se usan como admisibles para el dominio.

Modos de prueba: búsqueda, adversarial y partida.

Validación de escenarios y verificación independiente de soluciones.

Interfaz de consola con visualización del mapa, e interfaz gráfica con tres pestañas.

1.4 Metodología

El trabajo se organizó en tres fases con ciclos de retorno entre ellas.

La primera fue el análisis y diseño: estudiar el problema de búsqueda en espacios 2D, decidir la separación en capas, definir el espacio de estados y especificar la estructura del archivo de escenario.

La segunda fue la implementación: construir la carga y validación de escenarios, el modelo de estado, los siete algoritmos, los verificadores y las visualizaciones, acompañando cada pieza con pruebas.

La tercera fue la evaluación: ejecutar los algoritmos sobre escenarios donde divergen, variar parámetros como k en la búsqueda en haz o la profundidad en Minimax, y registrar las métricas para poder comparar.

El proceso no fue lineal. Varias veces hubo que volver a la implementación al detectar que un resultado no correspondía con lo esperado, y otras veces hubo que replantear el análisis al encontrar patrones que no se habían previsto.

2. Arquitectura

2.1 Separación en capas

Un sistema que mezcla la carga de archivos, la validación, los algoritmos y la presentación en un solo bloque es difícil de mantener y de probar. Por eso el proyecto está dividido en cinco capas, cada una con una responsabilidad definida.

La capa de escenario se ocupa de la realidad externa: leer el archivo JSON, validarlo y ofrecer acceso a sus datos.

La capa de dominio del juego modela las reglas: qué es un estado, qué acciones existen, cuándo termina la partida y cómo se evalúa una posición.

La capa de algoritmos contiene la lógica de búsqueda y decisión, escrita de forma genérica.

La capa de agentes conecta los algoritmos con el dominio del juego.

La capa de aplicación orquesta la ejecución, verifica los resultados y los presenta.

El flujo de dependencias va de arriba hacia abajo: la aplicación usa agentes, los agentes usan algoritmos, los algoritmos operan sobre el modelo de estado, y el modelo se construye a partir del escenario ya validado.

2.2 Capa de escenario

Se encarga de la relación con el mundo exterior. Sus tareas son leer el archivo JSON, comprobar que contiene todos los campos obligatorios, verificar que los datos sean consistentes y ofrecer métodos de lectura que no permitan modificaciones accidentales.

El punto importante es que la validación ocurre antes de que el escenario reemplace al que estaba cargado. Si el archivo tiene un error, el sistema lo rechaza completo y presenta la lista de problemas, sin quedar en un estado intermedio.

2.3 Capa de dominio del juego

Define el modelo de estado, las acciones y las transiciones. La clase GameState reúne los datos que describen una situación del juego; la enumeración Action lista las seis acciones posibles (cuatro movimientos, recoger y soltar); y los métodos de transición responden a la pregunta de a qué estado se llega al aplicar una acción válida.

Esta capa también contiene las funciones de utilidad que usa Minimax para valorar posiciones.

Una decisión de diseño que conviene señalar es que dos unidades no pueden ocupar la misma celda: la generación de movimientos impide entrar en una casilla ocupada.

2.4 Capa de algoritmos

Es la parte reutilizable del sistema. Contiene las funciones de búsqueda no informada, de costo uniforme, informada y adversarial, además de las heurísticas.

Los algoritmos no conocen el juego. Reciben un grafo, un diccionario de costos y, cuando corresponde, una función heurística. No hay posiciones ni dimensiones escritas en el código de los algoritmos. Esa independencia es la que permite que funcionen con cualquier escenario válido sin modificaciones.

Cada función devuelve un objeto de resultado con la misma forma: el camino o la ausencia de solución, el costo, la cantidad de nodos generados, la cantidad de nodos expandidos y el tamaño máximo alcanzado por la frontera.

2.5 Capa de agentes

Los agentes combinan un algoritmo con el estado del juego y las reglas del dominio. Un agente basado en BFS elige el siguiente movimiento usando esa búsqueda; uno basado en A* usa la heurística de Manhattan; uno táctico emplea Minimax. Esta capa es la que permite hablar de "una unidad que decide" sin que los algoritmos dejen de ser genéricos.

Los agentes pueden cargar escenarios JSON directamente, de modo que cualquier mapa nuevo funciona sin tocar código.

2.6 Capa de aplicación

Es la capa que coordina. El ejecutor de pruebas carga el escenario, construye el grafo y los costos, invoca el algoritmo que corresponda y reúne las métricas. El verificador comprueba por su cuenta que la solución sea válida. Los visualizadores presentan el mapa en consola y en HTML. El punto de entrada start.py concentra la lógica de despacho según el modo de prueba.

El flujo completo, desde el archivo hasta la respuesta, es el siguiente: se lee el JSON, se valida, se construye el objeto de escenario, el ejecutor arma el grafo y los costos,

se ejecuta el algoritmo, el verificador recalcula la solución y finalmente se presenta el resultado con sus métricas.

2.7 Visualización en consola

El mapa se dibuja en la terminal con símbolos por terreno, colores para distinguir bandos y marcadores para las bases y el recurso. La visualización se integra automáticamente en el punto de entrada, de modo que cada ejecución muestra el mapa antes de la búsqueda.

Conviene señalar una limitación conocida: el visualizador usa un símbolo por defecto para los terrenos que no tienen uno asignado, de manera que bosque y montaña se representan igual. El costo real de cada celda se consulta en el archivo del escenario.

2.8 Agentes integrados con escenarios

Los agentes que trabajan con archivos JSON cargan el escenario, validan su contenido, construyen el grafo a partir del mapa y ejecutan la búsqueda solicitada. Esto permite usar el mismo motor tanto desde los scripts de ejemplo como desde la interfaz gráfica.

3. Modelado del problema

3.1 Representación del estado

Un estado debe contener toda la información necesaria para tres cosas: generar los sucesores, decidir si se alcanzó la meta y calcular el costo de llegar hasta allí.

En navegación simple bastaría con la posición. Pero cuando hay varias unidades, turnos, posesión del recurso y acciones adversariales, esa representación deja de ser suficiente.

El estado en TacticalGrid se compone de cuatro elementos: las unidades, el recurso,

el turno actual y el historial de movimientos.

Las unidades se guardan en un diccionario que asocia el identificador con el objeto unidad. Cada unidad conoce su bando, su fila, su columna y si transporta el recurso.

El recurso se describe por su posición neutral y por el identificador de la unidad que lo transporta, o ninguno si está suelto. Se mantiene el invariante de que a lo sumo una unidad lo porta.

El turno indica qué bando debe actuar; después de cada acción se alterna.

El historial registra cada acción aplicada, lo que permite trazar la partida.

La razón de incluir estos cuatro elementos es directa: sin la posición no se pueden generar movimientos, sin el portador no se puede saber quién posee el recurso, y sin el turno no se puede determinar qué acciones son legales.

El caso más interesante es el que el enunciado señala de forma explícita: dos estados pueden tener una unidad en la misma casilla y aun así ser distintos, si en uno esa unidad transporta el recurso y en el otro no. Ese es el motivo por el cual la posesión se guarda como un atributo del estado y no se deduce de la posición.

3.2 Tamaño del espacio de estados

Para un mapa de m por n celdas y k unidades, una cota superior del número de estados es el producto de tres factores: las ubicaciones posibles de las k unidades, las k+1 opciones de portador (cada unidad o ninguna) y los dos turnos posibles.

En un mapa de 4 por 4 con dos unidades, la cota es 16 al cuadrado por 3 por 2, es decir 1536 estados. En la práctica el número es menor, porque no todas las celdas son transitables, porque dos unidades no pueden ocupar la misma casilla y porque la profundidad de búsqueda está acotada.

Este cálculo es útil porque muestra que el espacio es manejable para los tamaños que se usan en el proyecto.

3.3 Tipos de terreno

Cada celda tiene un tipo de terreno que determina si se puede entrar en ella y cuánto cuesta hacerlo. Los costos se leen del archivo del escenario, nunca de constantes escritas en el código.

Una consecuencia práctica de esa decisión es que cambiar el costo de un terreno en el JSON altera el comportamiento de los algoritmos que usan costos, sin necesidad de recompilar ni de modificar el programa. Este es justamente el tipo de prueba que se realiza durante la sustentación.

3.4 Unidades y posiciones

Una unidad tiene identificador, bando, fila, columna y un indicador de si transporta el recurso.

Se mantienen varios invariantes: cada unidad ocupa una sola celda, dos unidades no comparten celda, a lo sumo una porta el recurso, toda posición está dentro del mapa y toda posición corresponde a terreno transitable.

3.5 Recurso y bases

El recurso comienza en una posición neutral. Una unidad lo recoge al alcanzar esa celda y debe llevarlo a la base de su propio bando. Un equipo gana cuando su unidad portadora llega a su base con el recurso.

Esa regla genera el conflicto del juego: mientras un bando intenta entregar, el otro intenta bloquear o interceptar.

4. Sucesores y transiciones

4.1 Qué es un sucesor

Un sucesor de un estado es otro estado que se alcanza aplicando una acción válida. Para que un algoritmo de búsqueda funcione, el sistema debe poder responder tres

preguntas: qué acciones son legales en un estado, a qué estado se llega al aplicarlas y cuál es el costo del cambio.

4.2 Acciones válidas

Las acciones son cuatro movimientos, recoger el recurso y soltarlo. No todas son legales en todo estado.

Una unidad solo puede actuar si pertenece al bando del turno actual. Un movimiento requiere que la celda de destino sea transitable y que no esté ocupada por otra unidad. Recoger exige que la unidad esté sobre el recurso, que no lo transporte y que el recurso esté libre. Soltar exige que la unidad lo transporte y que no se encuentre sobre su propia base.

4.3 Transiciones

Aplicar una acción válida actualiza la posición de la unidad, o bien el atributo de portador y la posición del recurso, registra el movimiento en el historial y alterna el turno.

Durante la búsqueda se trabaja sobre copias del estado, de modo que explorar una rama no altera el estado original.

Vale la pena precisar cuatro observaciones. Los costos dependen del terreno de destino. Recoger y soltar no tienen costo; solo el movimiento lo tiene. Los costos son deterministas. Y el mismo movimiento cuesta siempre lo mismo, lo que permite comparar rutas.

4.4 Condiciones terminales

La partida termina cuando un bando gana, es decir, cuando una unidad que transporta el recurso se encuentra sobre la base de su propio equipo.

Adicionalmente se contempla el bloqueo: si el bando al que le corresponde actuar no tiene ninguna acción válida, la partida termina. Existe también una

comprobación específica para detectar si el portador del recurso quedó rodeado.

Las reglas más elaboradas, como la captura del portador por contacto, no están modeladas; se documentan como limitación en la sección correspondiente.

5. Validación de escenarios

5.1 Por qué validar

Los escenarios los suministra el profesor en formato JSON, y pueden llegar archivos con errores. La carga debe ser completa o no aplicarse: un escenario inválido se rechaza sin dejar el sistema en un estado parcialmente modificado.

5.2 Reglas implementadas

La validación se organiza en seis grupos que cubren las reglas del formato común.

El primero comprueba la presencia de todos los campos obligatorios: versión, mapa, tipos de terreno, matriz de terreno, bases, recurso, unidades, turno, configuración del juego y configuración de la prueba.

El segundo revisa el mapa: que filas y columnas sean enteros positivos y que la matriz tenga exactamente esas dimensiones.

El tercero revisa los terrenos: que todo tipo usado en la matriz esté definido en el catálogo y que todo terreno transitable tenga costo positivo.

El cuarto revisa las entidades: que bases, recurso y unidades estén dentro del mapa, que ninguna unidad se encuentre sobre una celda no transitable, que los identificadores sean únicos y que cada unidad pertenezca a un bando válido.

El quinto comprueba que el turno sea A o B.

El sexto verifica que, si hay portador declarado, corresponda a una unidad existente.

5.3 Ejecución

El validador recorre los seis grupos en orden y, si un grupo produce errores, detiene la revisión y devuelve la lista. La carga aplica la validación de forma atómica: si falla, se lanza un error con el detalle y no se reemplaza el escenario.

6. Función de utilidad

6.1 Qué es

Una función de utilidad asigna un número a cada estado, de manera que números mayores indiquen estados mejores desde la perspectiva de un bando. Se necesita porque Minimax debe comparar posiciones que no son terminales, donde todavía no hay un ganador.

6.2 Estrategias implementadas

El proyecto incluye tres estrategias, todas evaluadas desde el punto de vista de un bando.

La estrategia simple solo considera distancia. Si el bando ya ganó devuelve mil; si perdió, menos mil. Si alguna de sus unidades transporta el recurso, devuelve quinientos menos diez veces la distancia a su base. Si no, devuelve cien menos cinco veces la distancia mínima al recurso. Su costo es lineal en el número de unidades. Es simple, pero no modela bloqueos ni control del territorio.

La estrategia balanceada, que es la que se usa por defecto, combina cuatro componentes. La primera premia llevar el recurso a la base propia: parte de dos mil y resta veinte por cada paso de distancia. La segunda premia acercarse al recurso cuando nadie lo transporta: parte de quinientos y resta ocho por paso, sumando por cada unidad. La tercera es defensiva y crece con la distancia que separa al portador enemigo de su base, de modo que conviene cuando el rival está lejos de entregar. La cuarta es una bonificación menor por proximidad al centro del

mapa. Los valores de victoria y derrota son cinco mil y menos cinco mil. Su costo es cuadrático en el número de unidades.

La estrategia agresiva prioriza exclusivamente la captura y entrega del recurso. Usa valores extremos de diez mil y menos diez mil, y devuelve directamente una puntuación basada en la distancia a la base o al recurso, según el caso.

6.3 Uso en Minimax

Minimax alterna niveles de maximización y minimización. En los nodos donde decide el bando evaluado se toma el máximo valor entre los sucesores; en los nodos donde decide el rival se toma el mínimo. Cuando se alcanza la profundidad límite o un estado terminal, se llama a la función de utilidad.

Es importante insistir en un punto que suele malinterpretarse: los niveles de maximización y minimización no representan inteligencias distintas ni niveles de dificultad. Son roles dentro del árbol. El bando que minimiza la utilidad del otro está, con esa misma acción, favoreciendo su propio interés.

7. Búsqueda no informada

7.1 Búsqueda en anchura

BFS explora el espacio por niveles: primero el estado inicial, luego todos sus vecinos, después los vecinos de esos y así sucesivamente. Usa una cola en la que el primero en entrar es el primero en salir.

La consecuencia es que encuentra el camino con menos pasos. Lo que no hace es considerar costos: si dos rutas tienen el mismo número de pasos, cualquiera de las dos vale, sin importar cuánto cueste atravesarlas.

Su complejidad es lineal en nodos y aristas, tanto en tiempo como en el peor caso en espacio, porque la frontera puede contener buena parte del grafo.

En un mapa de m por n celdas, el número de nodos es el producto de ambos y el de aristas es aproximadamente cuatro veces ese producto, de modo que el tiempo resulta proporcional al tamaño del mapa.

BFS es la elección correcta cuando solo importa la cantidad de movimientos. Deja de serlo cuando los terrenos tienen costos distintos.

7.2 Búsqueda en profundidad

DFS explora una rama hasta el final antes de retroceder. Usa una pila en la que el último en entrar es el primero en salir.

No garantiza optimalidad: el camino que devuelve depende del orden en que se recorre a los vecinos. Su ventaja es el uso de memoria, que crece con la profundidad y no con la amplitud.

Su complejidad temporal en el peor caso es exponencial en la profundidad, y su espacio es lineal en ella.

DFS es útil cuando se necesita alguna solución con poca memoria, no cuando se necesita la mejor.

8. Búsqueda informada

8.1 Búsqueda de costo uniforme

UCS minimiza el costo acumulado en lugar del número de pasos. Para lograrlo usa una cola de prioridad ordenada por ese costo, en lugar de una cola simple.

El invariante que sostiene su optimalidad es que el primer estado que se extrae del objetivo tiene costo mínimo, siempre que los costos de las transiciones sean positivos.

Su complejidad temporal incluye el costo logarítmico del montículo, y su espacio es lineal en el número de nodos.

La comparación con BFS es la que da lugar al primer resultado interesante del proyecto. Si el mapa tiene terrenos de distinto costo, BFS puede devolver un camino más corto en pasos pero más caro, mientras que UCS devuelve uno más largo pero más barato. Ambos son correctos según su propio criterio; lo importante es saber cuál criterio se necesita.

*8.2 A**

A* combina el costo real recorrido con una estimación del costo restante. Para cada estado calcula la suma de ambos valores y expande primero el de suma menor.

La garantía de optimalidad depende de la heurística. Si la estimación nunca sobrestima el costo real, A* encuentra el camino de costo mínimo. Esa propiedad se denomina admisibilidad.

La ventaja frente a UCS es que la estimación orienta la expansión hacia el objetivo, de manera que se expanden menos estados para llegar al mismo resultado.

El proyecto implementa cuatro heurísticas y documenta dos más.

La distancia Manhattan suma las diferencias absolutas de fila y columna. Para un dominio con movimiento en cuatro direcciones es admisible, porque no es posible llegar en menos pasos que esa distancia. No observa el mapa, así que subestima cuando hay obstáculos que obligan a rodear.

La segunda heurística propia multiplica la distancia Manhattan por el costo mínimo de un paso transitable. Si todo paso cuesta al menos ese mínimo, el costo restante es al menos la distancia en pasos por ese mínimo, y por lo tanto la estimación sigue siendo admisible. Es más informativa que Manhattan en mapas con terrenos caros.

La tercera, basada en un recorrido en anchura desde el objetivo, calcula para cada celda el número real de pasos hasta la meta respetando los muros y lo multiplica por el costo mínimo. Relaja los costos del terreno pero conserva los obstáculos. Es

admisible y consistente, y domina a Manhattan en el sentido de que nunca estima menos, por lo que expande igual o menos estados. Su costo de preparación es un recorrido completo del grafo, que se paga una sola vez.

La heurística nula devuelve cero y sirve como referencia: convierte a A* en UCS, lo que permite medir cuánto aporta una heurística concreta.

Las distancias euclidiana y Chebyshev están implementadas y documentadas, pero se advierte que no son admisibles para movimiento en cuatro direcciones, porque suponen diagonales. Usarlas rompería la garantía de optimalidad.

8.3 Búsqueda en haz

La búsqueda en haz es una variante de A* con memoria limitada. Después de generar los sucesores, conserva únicamente los k mejores según su valor estimado y descarta el resto.

Ese recorte reduce la memoria a un valor proporcional a k, pero elimina la garantía de optimalidad: un estado descartado en un nivel podría haber conducido a una mejor solución más adelante. El parámetro k controla el equilibrio: con k igual a uno el comportamiento se aproxima a una búsqueda codiciosa, y a medida que k crece se aproxima a A*.

9. Búsqueda adversarial

9.1 Minimax

Minimax se usa cuando el entorno deja de ser pasivo. La pregunta ya no es cuál es el mejor camino, sino cuál es la mejor acción teniendo en cuenta que el adversario responderá.

El algoritmo construye un árbol de jugadas hasta una profundidad límite. En los niveles propios se elige el mayor valor y en los niveles del rival el menor, suponiendo que el rival juega de forma óptima. Al llegar a un nodo hoja se evalúa

con la función de utilidad.

Su complejidad es exponencial en la profundidad, ya que el número de nodos crece con el factor de ramificación elevado a la profundidad. Con un factor de cuatro y profundidad ocho se superan los sesenta y cinco mil nodos, y las cifras crecen muy rápido a partir de ahí.

9.2 Poda alfa-beta

Alfa-beta produce exactamente el mismo resultado que Minimax, pero explora menos ramas.

La idea es llevar dos valores durante la exploración: alfa, el mejor valor que el maximizador ya puede garantizarse, y beta, el mejor que el minimizador ya puede garantizarse. Cuando alfa alcanza o supera a beta, la rama en curso ya no puede cambiar la decisión en el nivel superior y se abandona.

En el mejor caso la complejidad baja a la raíz cuadrada del exponente, y en el peor caso coincide con Minimax. Cuánto se gana depende del orden en que se examinan los sucesores: si se examinan primero los movimientos prometedores, la poda es mucho mayor.

Ese último punto es relevante porque permite comprobarlo experimentalmente: al invertir el orden de generación de sucesores, la decisión final no cambia, pero la cantidad de nodos podados sí.

10. Experimentos

10.1 Divergencia entre BFS y UCS

El escenario de divergencia muestra el efecto de los costos. Es un mapa de tres filas por cinco columnas con dos tipos de terreno: camino, con costo uno, y pasto, con costo cinco. La prueba lleva la unidad A1 desde la esquina superior izquierda hasta la esquina superior derecha.

BFS encuentra una ruta de cuatro movimientos recorriendo la fila superior. Esa ruta es la más corta en pasos, pero atraviesa tres celdas de pasto, de modo que su costo real es dieciséis.

UCS encuentra una ruta de ocho movimientos que baja hasta la fila inferior, la recorre y sube. Es el doble de larga, pero todas sus celdas son de camino, y su costo total es ocho.

El resultado confirma que ambos algoritmos son correctos según su criterio y que, en presencia de costos distintos, la ruta más corta puede no ser la más económica.

10.2 Eficiencia de A frente a UCS*

Sobre el escenario de terrenos costosos, un mapa de cinco por cinco, ambos algoritmos encuentran el mismo camino de costo ocho.

La diferencia está en el trabajo realizado: UCS expande quince nodos y A* con la heurística basada en la distancia real expande nueve. La heurística concentra la exploración en la zona que conduce al objetivo y evita revisar alternativas que el costo real mantiene baratas pero que están lejos de la meta.

La comparación entre heurísticas sobre el mismo escenario es igualmente informativa. Con la heurística nula, A* se comporta como UCS y expande quince nodos. Con Manhattan o con su variante ponderada, expande nueve. La diferencia cuantifica el aporte de la información heurística.

10.3 Memoria y calidad en la búsqueda en haz

Se ejecutó la búsqueda en haz con valores de k iguales a uno, dos, cuatro y ocho sobre los escenarios del proyecto, y los resultados quedaron registrados en un archivo de resultados.

En el escenario de divergencia, todos los valores de k encuentran la misma solución de costo ocho, porque la frontera rara vez contiene más de un estado prometedor.

El experimento muestra también el límite del análisis: en mapas pequeños y con pocos caminos alternativos, restringir la frontera no cambia el resultado. Para observar el caso en que un k pequeño descarta una ruta que luego habría sido conveniente se necesitaría un escenario con decisiones ambiguas, donde varios estados tengan valores estimados parecidos. Los escenarios actuales no lo evidencian, y eso se declara explícitamente en lugar de presentar un resultado que no se observó.

10.4 Casos mínimos

El script de casos mínimos reproduce automáticamente los diez casos solicitados y compara lo esperado con lo obtenido.

El primer caso contrasta BFS y DFS sobre el mismo mapa: ambos encuentran solución, pero recorren el espacio de forma distinta.

El segundo corresponde a la divergencia entre ruta corta y ruta económica, descrita en 10.1.

El tercero usa un mapa con barreras donde el objetivo parece cercano pero obliga a rodear, lo que muestra cómo una heurística que no ve los obstáculos pierde precisión.

El cuarto compara heurísticas sobre el mismo escenario, con los resultados descritos en 10.2.

El quinto varía k en la búsqueda en haz, con los resultados descritos en 10.3.

El sexto evidencia el estado compuesto. Sobre el escenario básico, una unidad en la misma posición recibe utilidad ochenta y cinco cuando no transporta el recurso y mil cuando lo transporta, con la estrategia simple. La posición física es idéntica; lo que cambia es la información adicional del estado.

El séptimo muestra que la acción elegida por Minimax no coincide necesariamente con avanzar directamente hacia el recurso, porque la decisión pondera la respuesta

del adversario.

El octavo compara Minimax y alfa-beta sobre el mismo estado: ambos eligen la misma acción y obtienen el mismo valor, pero alfa-beta genera y evalúa menos nodos y registra podas.

El noveno invierte el orden de generación de sucesores y observa cómo cambia la cantidad de poda.

El décimo varía la profundidad de uno a cuatro y registra cómo cambia el valor obtenido y cuántos nodos se generan, lo que ilustra el efecto del horizonte de búsqueda.

11. Complejidad

La siguiente tabla resume el costo temporal y espacial de cada algoritmo, junto con su condición de optimalidad.

BFS usa tiempo lineal en nodos y aristas y espacio lineal en nodos, y es óptimo en número de pasos.

DFS usa tiempo exponencial en la profundidad en el peor caso y espacio lineal en la profundidad, y no es óptimo.

UCS usa tiempo lineal por el logaritmo del número de nodos, debido al montículo, y espacio lineal. Es óptimo en costo.

A* tiene la misma cota que UCS, pero en la práctica expande menos cuando la heurística es informativa. Es óptimo si la heurística es admisible.

La búsqueda en haz usa tiempo proporcional a k por nivel y espacio proporcional a k, y no es óptima.

Minimax usa tiempo exponencial en la profundidad y espacio lineal en ella, y es óptimo bajo el supuesto de juego perfecto del adversario.

Alfa-beta reduce el exponente en el mejor caso, mantiene la misma cota en el peor, y conserva la optimalidad de Minimax.

Aplicado a los escenarios del proyecto, el análisis muestra que en mapas de cuatro por cuatro todas las alternativas son baratas y las diferencias son menores. En mapas grandes, en cambio, la elección importa: A* reduce el trabajo frente a UCS, y la búsqueda en haz lo reduce mucho más a costa de perder la garantía de optimalidad.

12. Limitaciones

La terminación de la partida en casos sin ganador es una limitación conocida. La partida termina al detectar un ganador, un bloqueo o al alcanzar el límite de turnos. En tableros donde la función de utilidad no conduce a entregar el recurso, la partida puede agotar el límite sin ganador. Eso refleja una limitación del diseño de la utilidad, no del motor de búsqueda.

Las reglas de bloqueo son básicas. Se contempla el caso en que el bando del turno no tiene acciones y el caso del portador rodeado, pero no se modelan interacciones más ricas, como la captura del portador por contacto.

Las heurísticas están implementadas y justificadas, pero su comportamiento depende del tipo de mapa. La heurística basada en distancia real gana cuando hay obstáculos; en mapas abiertos coincide con Manhattan y solo añade el costo de preparación.

Los escenarios son estáticos y de información perfecta. El mapa no cambia durante la búsqueda y los algoritmos lo conocen por completo.

No hay planificación multiagente. Cada unidad se planifica de forma independiente y no coordina sus movimientos con las demás.

Finalmente, el visualizador de consola no distingue todos los tipos de terreno, como se señaló en la sección de arquitectura.

13. Uso de inteligencia artificial

De acuerdo con el enunciado, el uso de herramientas de inteligencia artificial generativa debe declararse y ser trazable tanto en la documentación de los métodos como en este documento.

El uso está declarado en cada método relevante mediante los campos de documentación exigidos: propósito, precondiciones, postcondiciones, complejidad, y los campos específicos de uso de IA, intervención y validación.

En términos generales, la herramienta se empleó como apoyo durante el desarrollo: borradores de documentación, esbozos de pseudocódigo y redacción inicial de secciones de este informe.

Los resultados numéricos que aparecen en el documento no provienen de estimaciones: se obtuvieron ejecutando el código sobre los escenarios del proyecto. La suite de pruebas, que cubre búsqueda, heurísticas, estado del juego, algoritmos adversariales, reglas del recurso, validación y verificación, se ejecuta como comprobación de que el comportamiento descrito corresponde al implementado.

14. Referencias

Russell, S. y Norvig, P. Artificial Intelligence: A Modern Approach, cuarta edición, 2020.

Nilsson, N. Artificial Intelligence: A New Synthesis, 1998.

Hart, P., Nilsson, N. y Raphael, B. A Formal Basis for the Heuristic Determination of Minimum Cost Paths, 1968.

Korf, R. Real-Time Heuristic Search, 1990.

Goodrich, M., Tamassia, R. y Mount, D. Data Structures and Algorithms in Python, 2011.

Sistemas Inteligentes I

Cristian Camilo Osorio Granada

2026